

System 14: In-Memory Pub/Sub Message Broker (Kafka Lite)

Zero-Prerequisite Intuition: The "Newspaper Subscription" Metaphor What is a Publish/Subscribe (Pub/Sub) message broker, and why did the software industry invent it? Imagine a news agency that prints financial news. If the news agency had to personally call 50,000 individual readers on the telephone every time a story broke, the journalists would spend 100% of their day dialing phones. If 5,000 new readers sign up, the journalists are completely overwhelmed.

Instead, the agency prints newspapers and drops them into a central **Newsstand (The Topic)**.

- **Publishers (Journalists):** Drop articles into the newsstand and immediately walk away.
- **Subscribers (Readers):** Walk up to the newsstand whenever they are free, read the articles at their own speed, and remember which page they finished reading (**The Offset**).

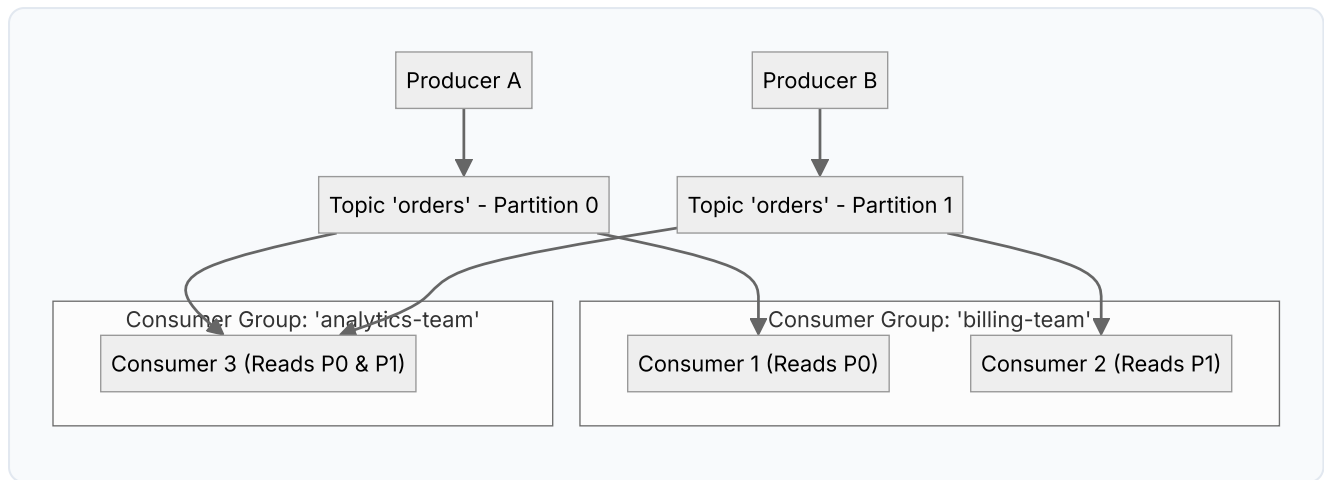
In microservices (Amazon, Netflix, Uber), Order Services don't call Billing, Shipping, Analytics, and Fraud directly. They publish an `OrderPlaced` event to a message broker, and each downstream team consumes it independently!

This chapter designs an **In-Memory Pub/Sub Message Broker** (a simplified Apache Kafka engine) in pure Python.

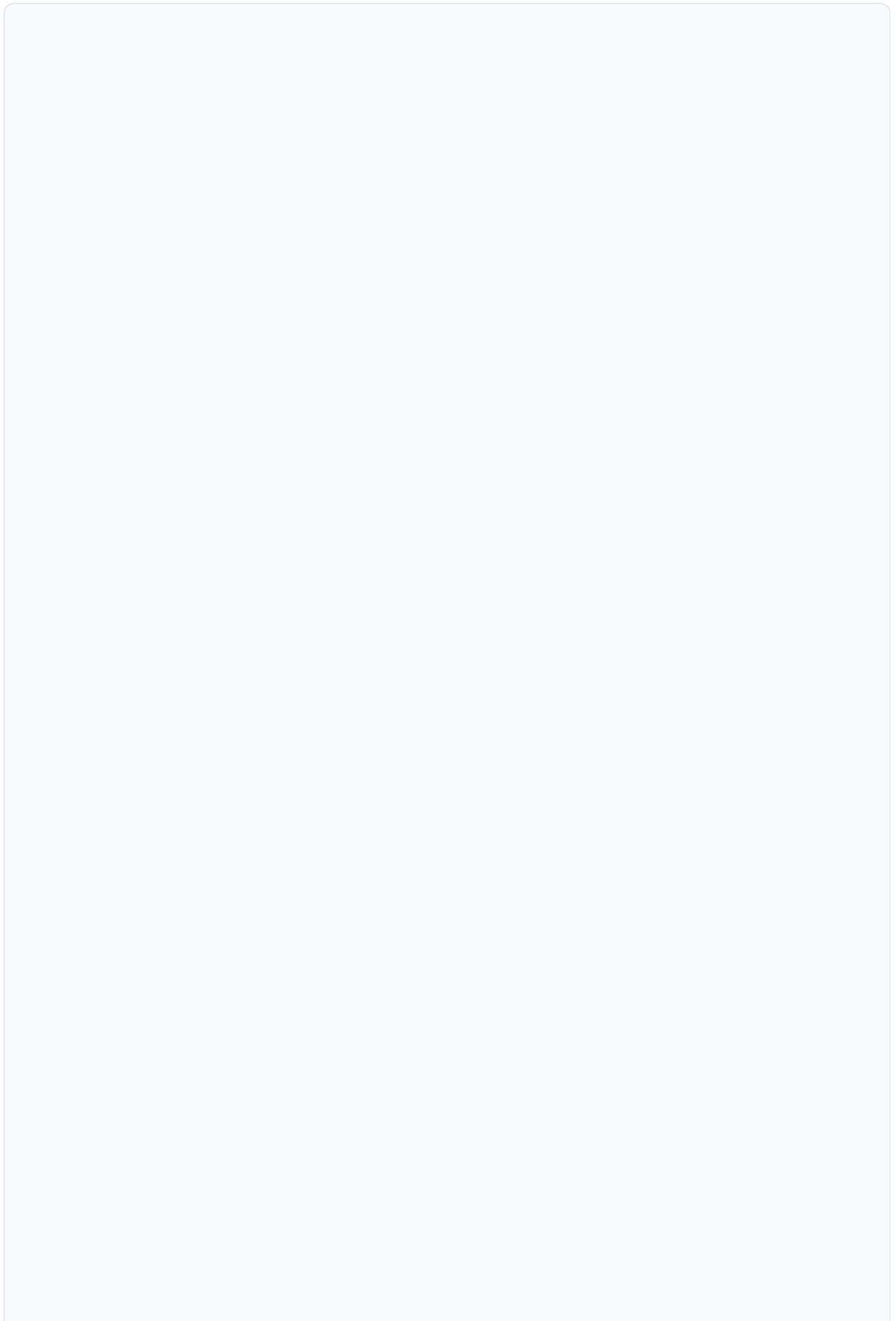
1. Requirements & Core Concepts

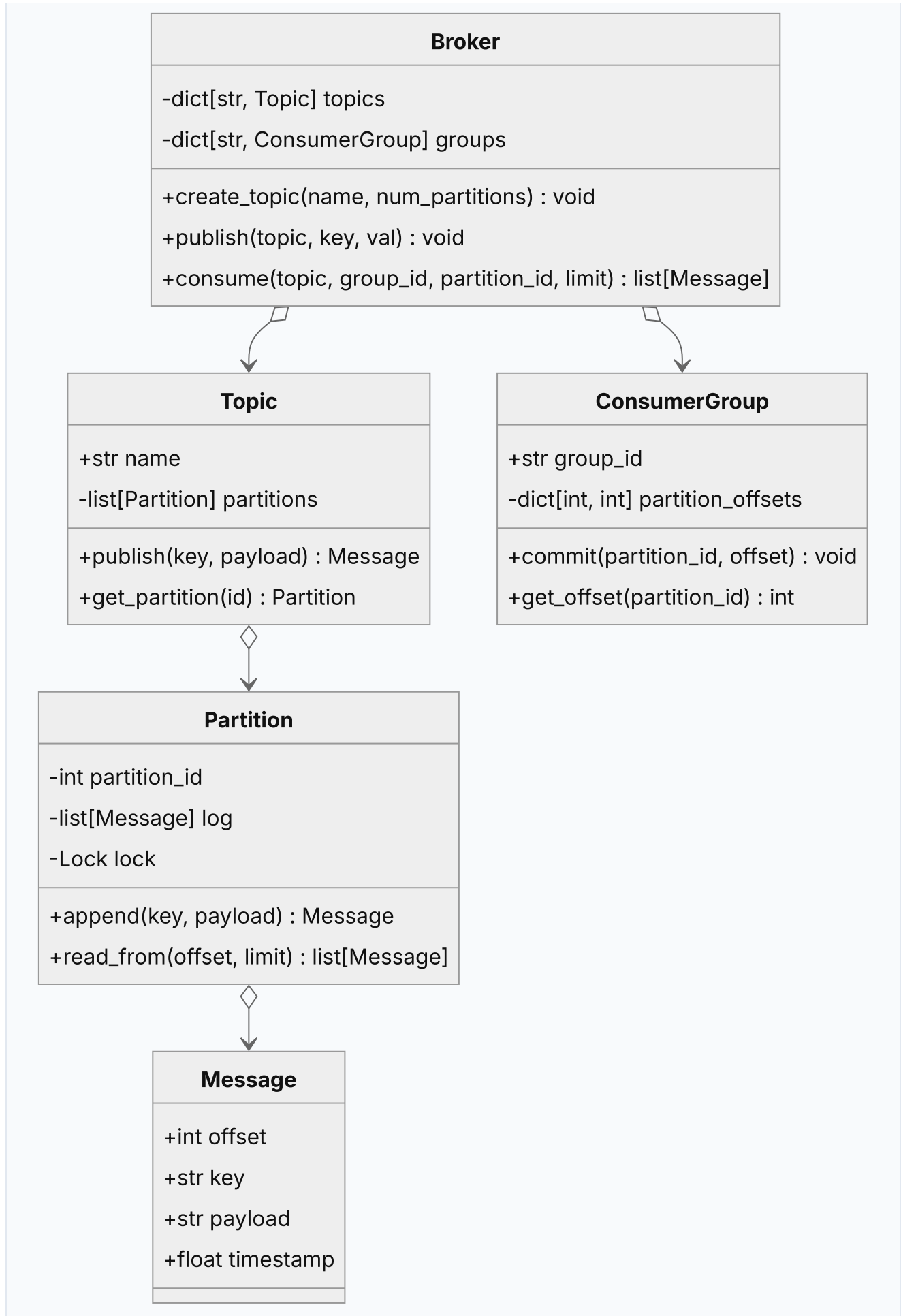
Core Primitives Defined

1. **Message:** An immutable payload with a `key`, `value`, `timestamp`, and assigned `offset`.
2. **Topic:** A named logical stream of messages (e.g., `user-signups`).
3. **Partition:** A strictly ordered, append-only log within a topic that guarantees message order.
4. **Consumer Group:** A collection of consumers cooperating to read data. Each message sent to a topic is delivered to **one consumer instance within each subscribing consumer group**.



2. Design Pattern & Class Hierarchy





3. Complete Python Implementation

4. Verification: Multi-Consumer Group Isolation

Both consumer groups read the exact same data without interfering with each other's reading progress!